



CIBERSEGURIDAD EN ENTORNOS DE LAS TECNOLOGÍAS DE LA INFORMACIÓN

Puesta en Producción Segura

TEMA 2

Determinación del nivel de seguridad requerido por aplicaciones
Alberto Diéguez Álvarez

Contenido

1.- Descripción de la tarea.....	3
Caso práctico	3
Apartado 1: Responde a las siguientes cuestiones.....	4
Apartado 2: Crea el entorno de pruebas	5
Apartado 3: Pruebas de vulnerabilidades.....	11
Apartado 4: Preguntas finales.....	17

1.- Descripción de la tarea.

Caso práctico



[Pixabay](#) (Dominio público)

Julián está preocupado porque necesita crear un aplicativo web que interactuara con una base de datos y sabe que uno de los principales riesgos es la entrada de datos de usuarios ya que podrían provocar acceso a información que no debiera de la base de datos. Ha buscado información sobre vulnerabilidades de este tipo y ha decidido que debe aprender cómo desarrollar de forma más segura

Para ello crea el entorno de pruebas Damn Vulnerable Web Application (<https://github.com/digininja/DVWA>) donde podrá configurar distintos niveles de seguridad, ver cómo se refleja en el código y probar distintos tipos de ataque y ver de qué manera se comporta el aplicativo y sobre todo ver que recibe la base de datos.

Va a probar ataques de tipo Injection o inyección que es uno de los principales riesgos identificados en OWASP.

¿Qué te pedimos que hagas?

Apartado 1: Responde a las siguientes cuestiones

Buscar información sobre la vulnerabilidad **CVE-2023-39417** y rellenar la siguiente tabla.

Información sacada de: [INCIBE](#)

Breve descripción	EN EL SCRIPT DE EXTENSIÓN, se encontró una vulnerabilidad de inyección SQL en PostgreSQL si usa @extowner@, @extschema@ o @extschema:...@ dentro de una construcción de cotización (cotización en dólares, " o ""). Si un administrador ha instalado archivos de una extensión vulnerable, de confianza y no empaquetada, un atacante con privilegios CREATE de nivel de base de datos puede ejecutar código arbitrario como superusuario de arranque.		
Impacto	Vector 3.x: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H Puntuación base 3.x: 8.80 Gravedad 3.x: ALTA		
Productos y versiones vulnerables	CPE	DESDE	HASTA
	cpe:2.3:a:postgresql:postgresql:*.***.*.*.*	11.0 (incluyendo)	11.21 (excluyendo)
	cpe:2.3:a:postgresql:postgresql:*.***.*.*.*	12.0 (incluyendo)	12.16 (excluyendo)
	cpe:2.3:a:postgresql:postgresql:*.***.*.*.*	13.0 (incluyendo)	13.12 (excluyendo)
	cpe:2.3:a:postgresql:postgresql:*.***.*.*.*	14.0 (incluyendo)	14.9 (excluyendo)
	cpe:2.3:a:postgresql:postgresql:*.***.*.*.*	15.0 (incluyendo)	15.4 (excluyendo)
	cpe:2.3:a:redhat:software_collections:~.***.*.*.*		
	cpe:2.3:o:redhat:enterprise_linux:8.0:~.***.*.*		
	cpe:2.3:o:redhat:enterprise_linux:9.0:~.***.*.*		
	cpe:2.3:o:debian:debian_linux:8.0:~.***.*.*.*		
	cpe:2.3:o:debian:debian_linux:11.0:~.***.*.*		
cpe:2.3:o:debian:debian_linux:12.0:~.***.*.*			
Posibles soluciones	Hay varias soluciones: PostgreSQL: https://www.postgresql.org/support/security/CVE-2023-39417/ La solución que mas se repite es actualizar PostgreSQL a nuevas versiones parcheadas o eliminar extensiones que usen @extraowner@, @extschema@		

Buscar información del requisito **ASVS v4.0.3-5.3.4** (**ASVS versión 4.0.3 capítulo 5, apartado 3, requisito 4**) y rellena la siguiente tabla:

- Link al archivo:

<https://raw.githubusercontent.com/OWASP/ASVS/v4.0.3/4.0/OWASP%20Application%20Security%20Verification%20Standard%204.0.3-es.pdf>

Breve descripción	Verifique que la selección de datos o las consultas de base de datos (por ejemplo, SQL, HQL, ORM, NoSQL) utilizan consultas parametrizadas, ORM, marcos de entidades o están protegidas de los ataques de inyección de base de datos. (C3)
Niveles a los que debe aplicarse	L1, L2, y L3
Categoría CWE	89

Apartado 2: Crea el entorno de pruebas

Hoy en día la mayoría de los equipos de desarrollo utilizan contenedores para montar los entornos de desarrollo. En la página web del proyecto de DVWA (<https://github.com/digininja/DVWA>) explican como instalar la aplicación con Docker, que es una de las herramientas más habituales para trabajar con contenedores.

Sigue las siguientes instrucciones:

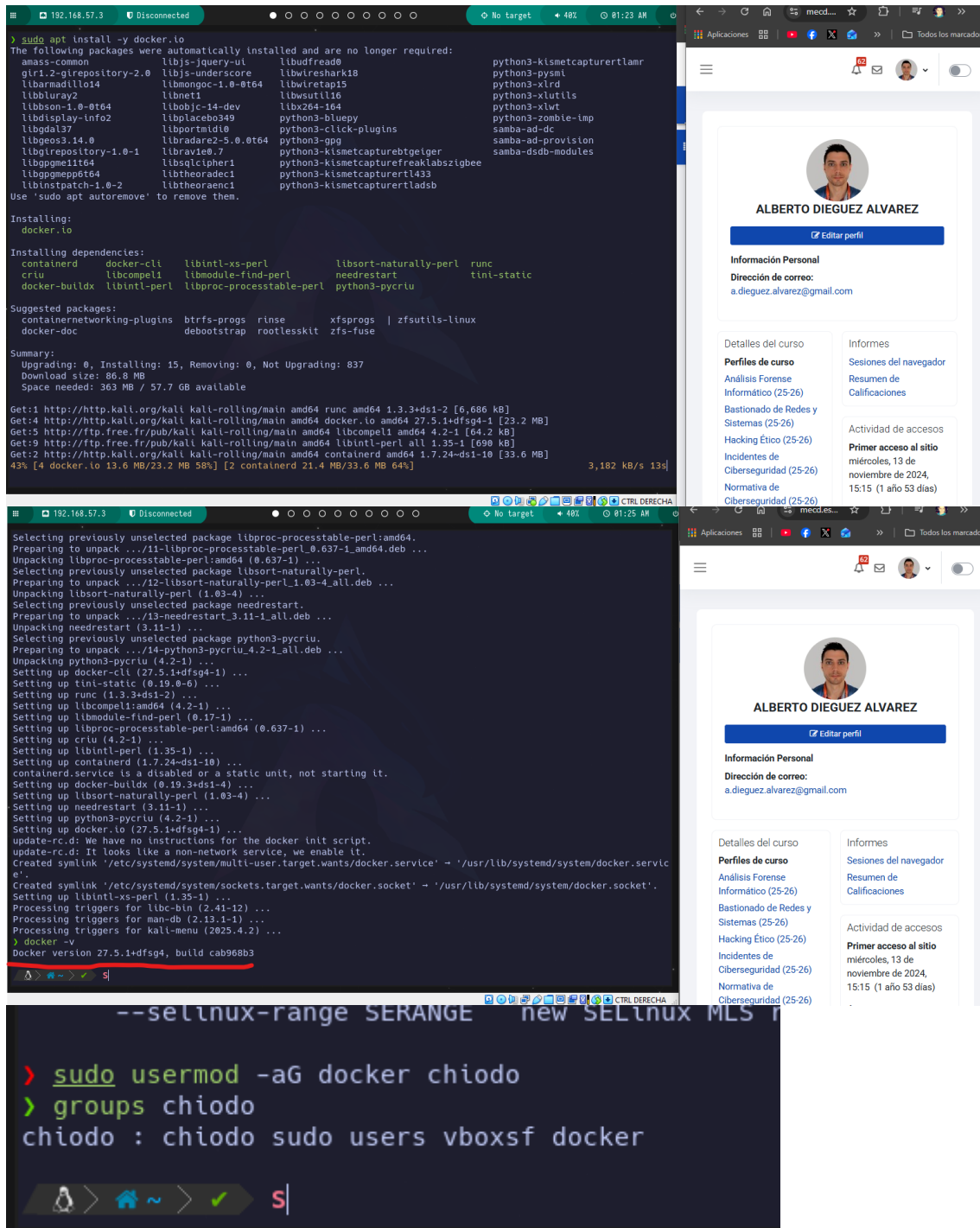
1. Instalar Docker y si es necesario docker compose (en algunas versiones ya viene con docker). En la página oficial de Docker explican cómo instalarlo

Uso una imagen de Kali Linux configurada por mí, dejo mi repositorio:

<https://github.com/alberto-diequez/ansible-auto-bspwm>

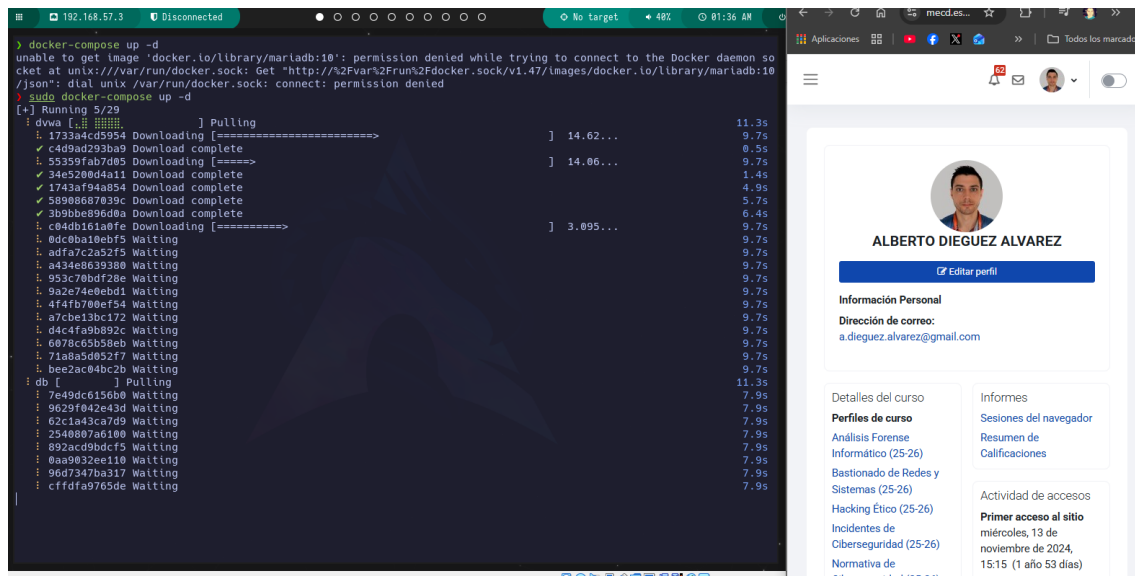
Instalo Docker:

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable docker --now
sudo usermod -aG docker $USER
groups $USER
```

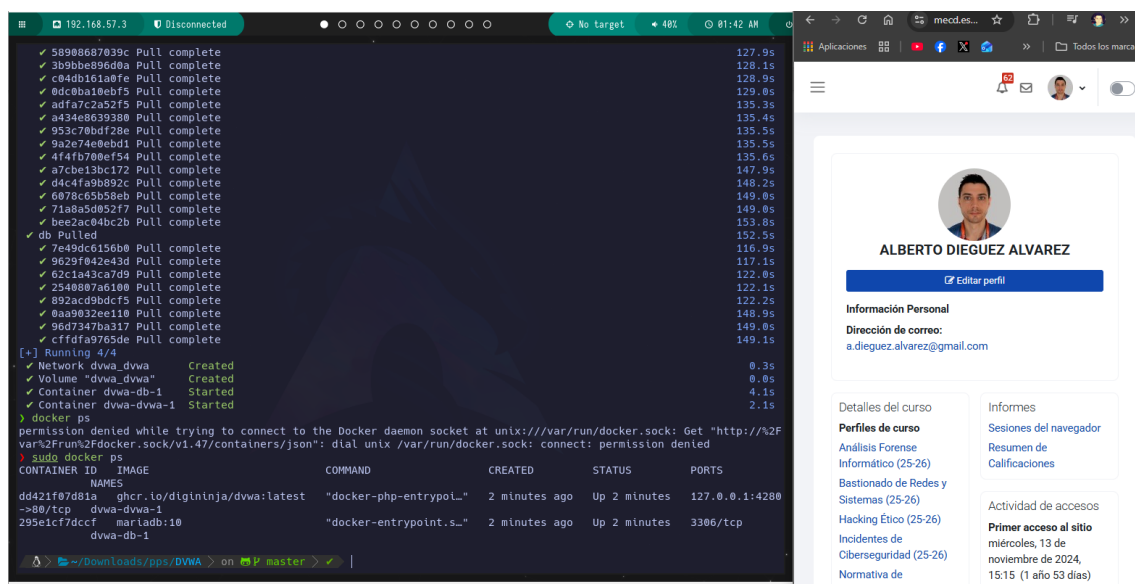


Instalo Docker-compose:

sudo apt install docker-compose

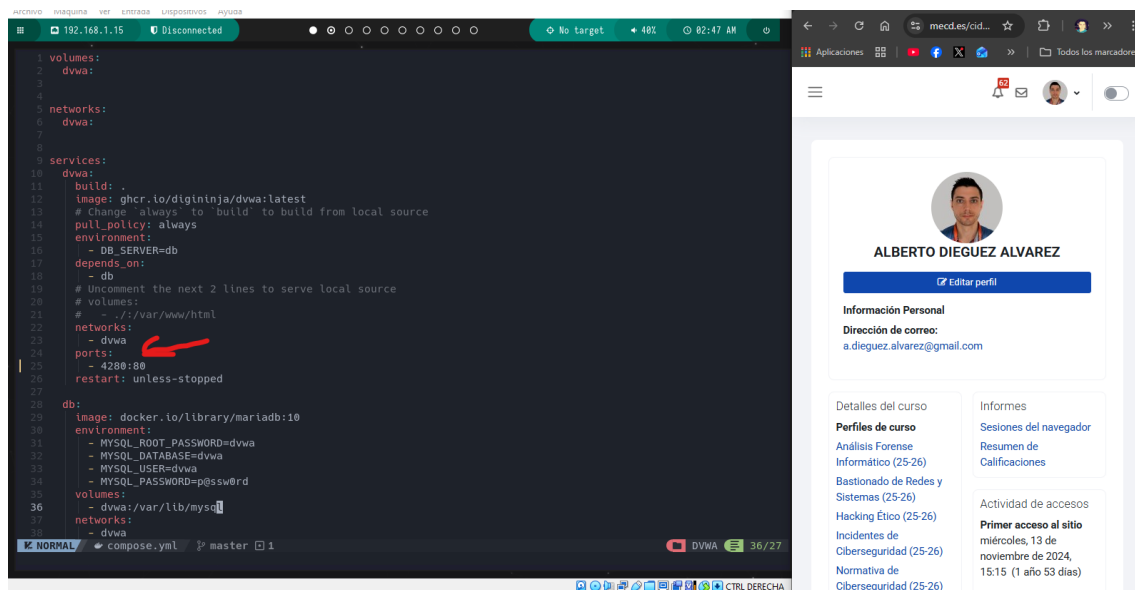


Compruebo la instalación con Docker ps.

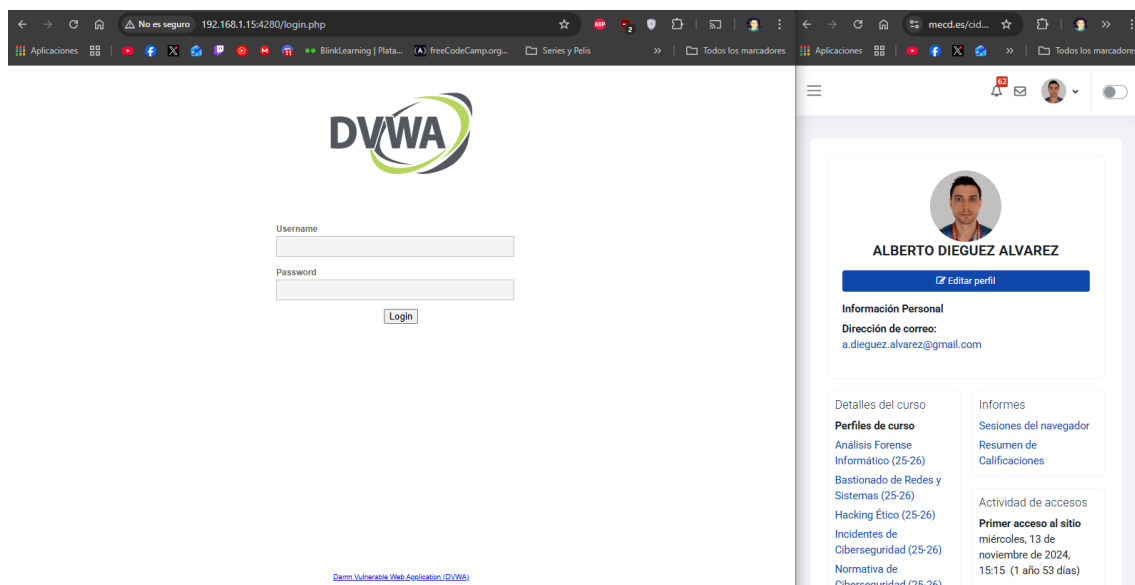


- Una vez instalado, para acceder a DVWA:
 - Abrir un navegador con la dirección <http://localhost:4280> El usuario es admin y la clave es admin

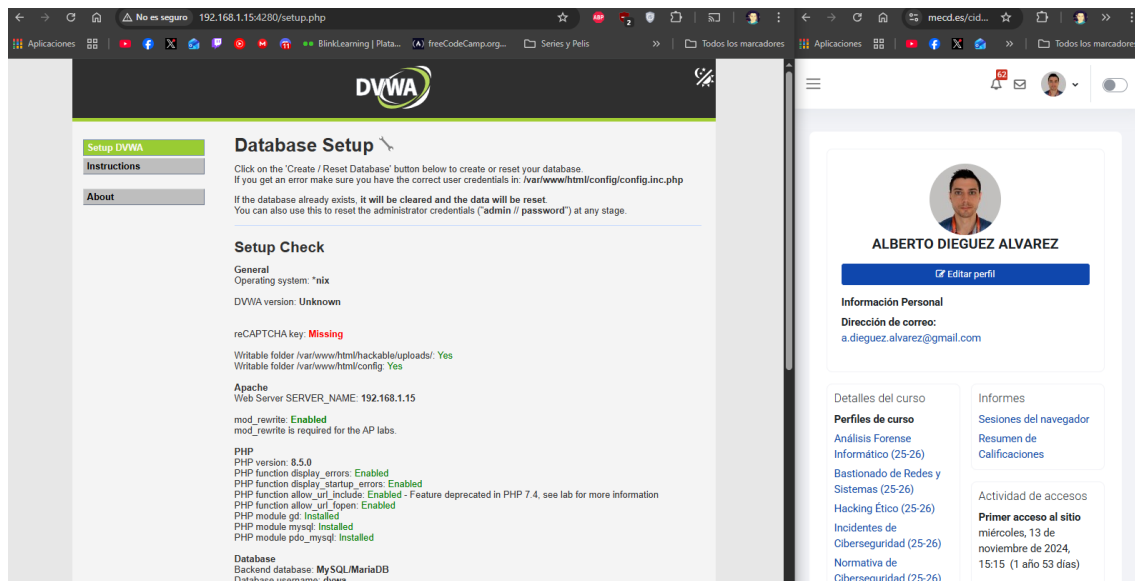
Para poder entrar desde Windows y facilitar algunas tareas, modifiko compose.yml para quitar el host de ports en dwa y lo dejo como en la imagen para luego poner en modo puento la VM y poder acceder desde Windows.



Entro a DVWA desde Windows:

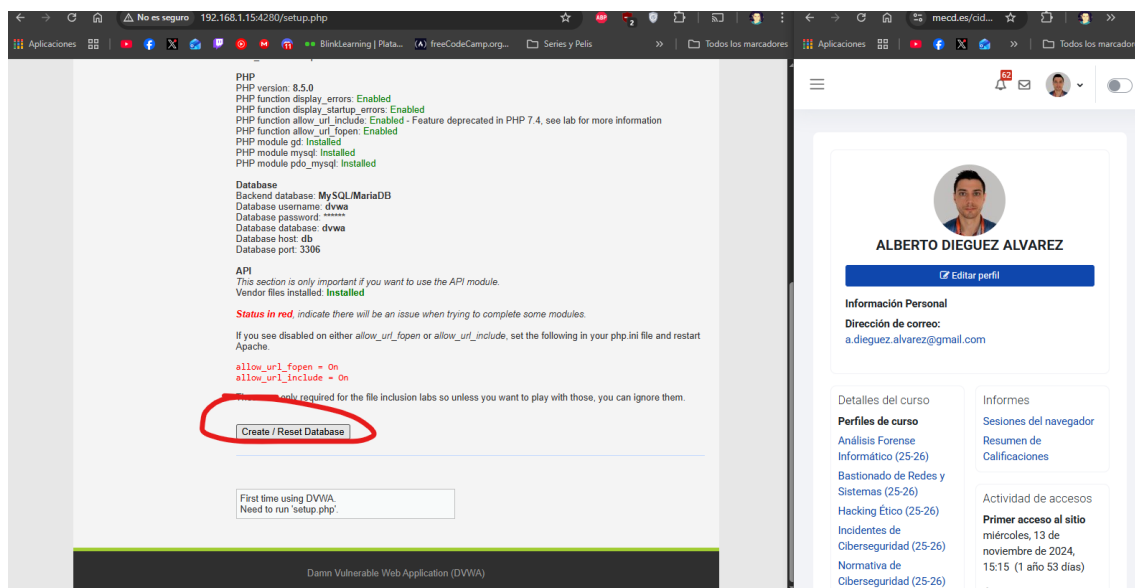


Accedo con admin/admin:



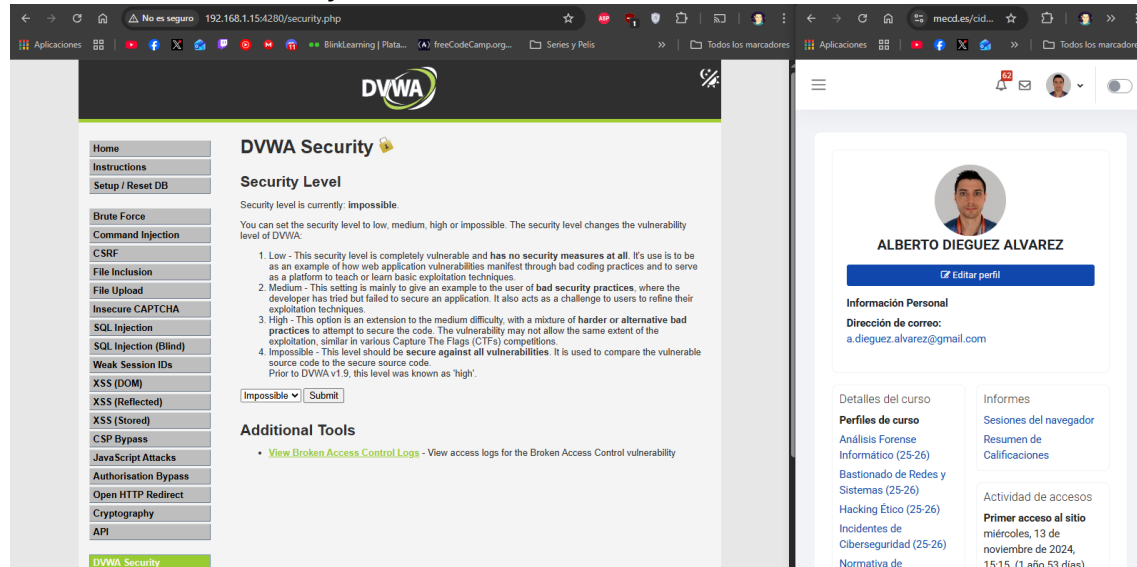
- La primera vez que se abre la aplicación es necesario pulsar en el botón Create/Reset Database

Ejecuto el botón.



- A partir de ese momento el usuario es **admin** y la clave es **password**
- Para cambiar el nivel de seguridad debes ir a **DVWA Security**

Entro a DVWA Security:

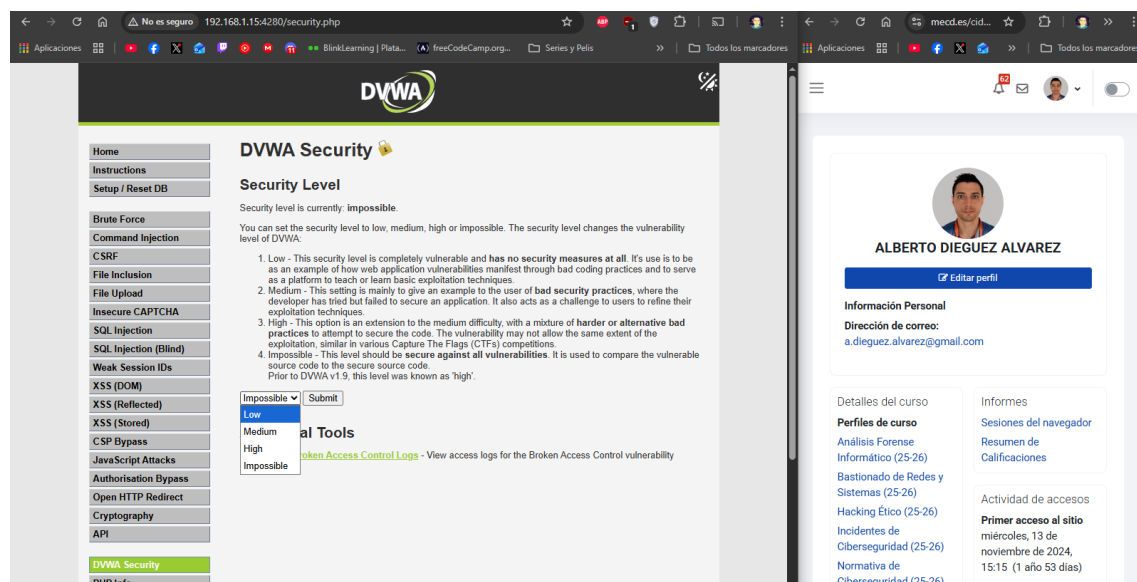


Apartado 3: Pruebas de vulnerabilidades.

Procederemos a realizar distintas pruebas de SQL Injection

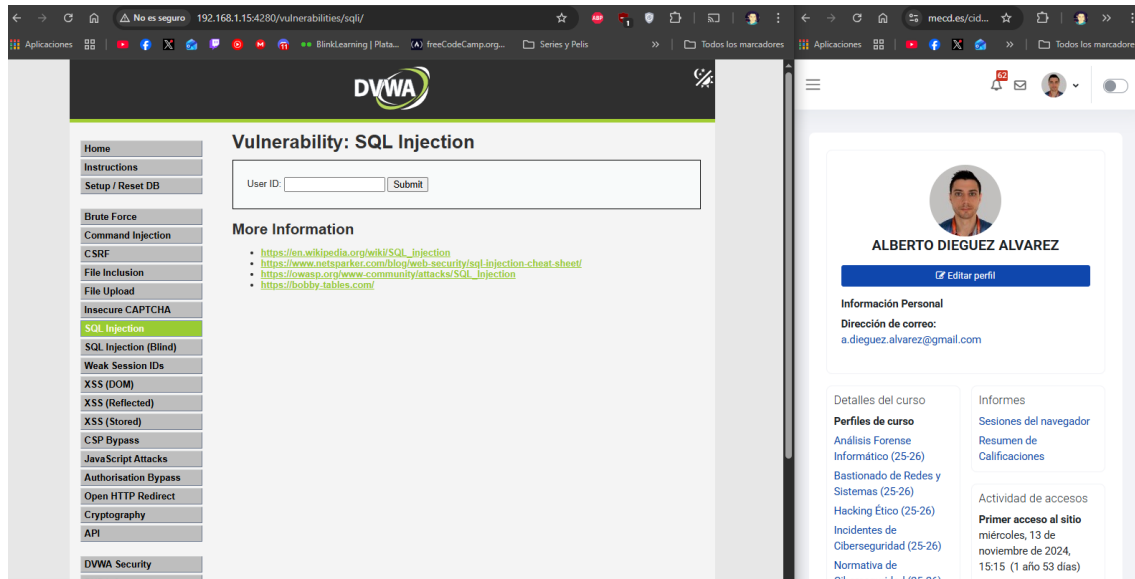
1. Accederemos en modo Low

Activo el modo Low en DVWA Security.



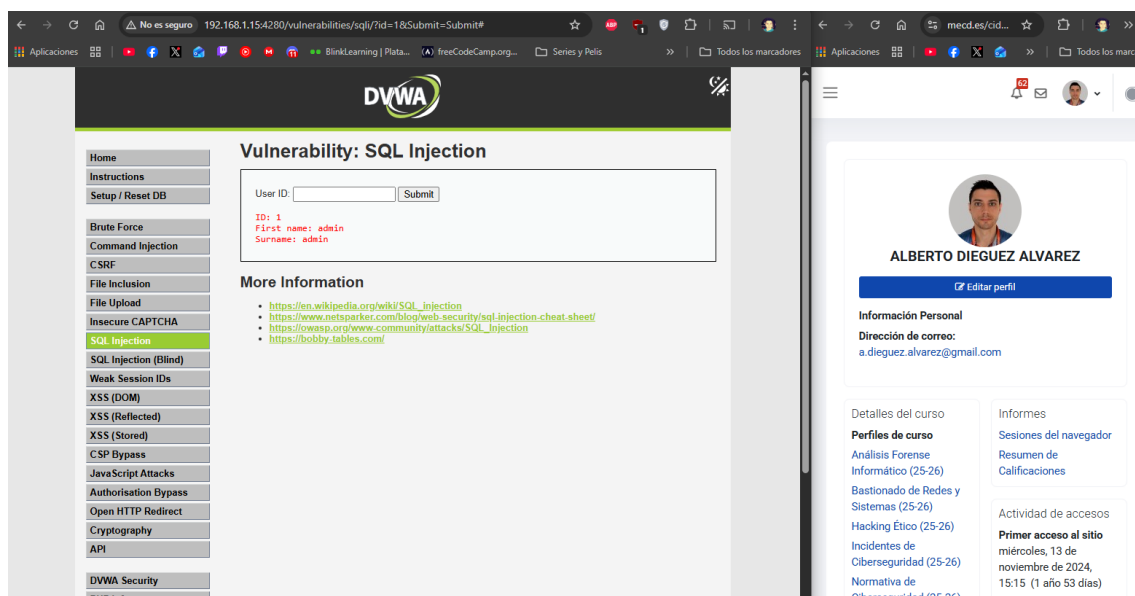
2. Pulsar en el menú lateral **SQL Injection**

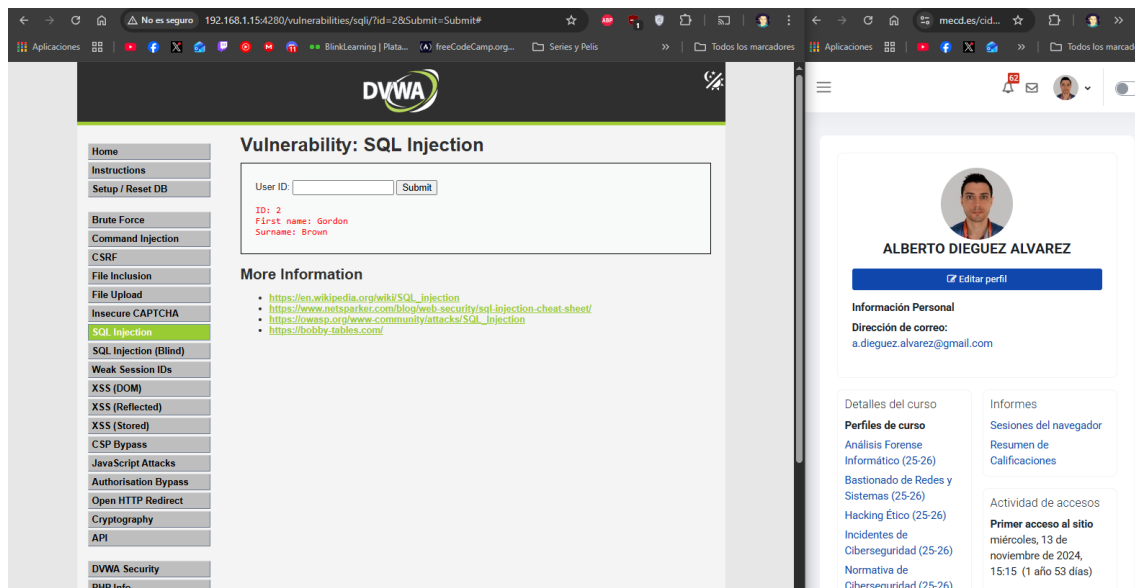
Entro en SQL Injection:



3. Primero comprobaremos cómo se comporta nuestro aplicativo cuando le damos IDs de usuarios (puedes probar con 1, 2, 3..)

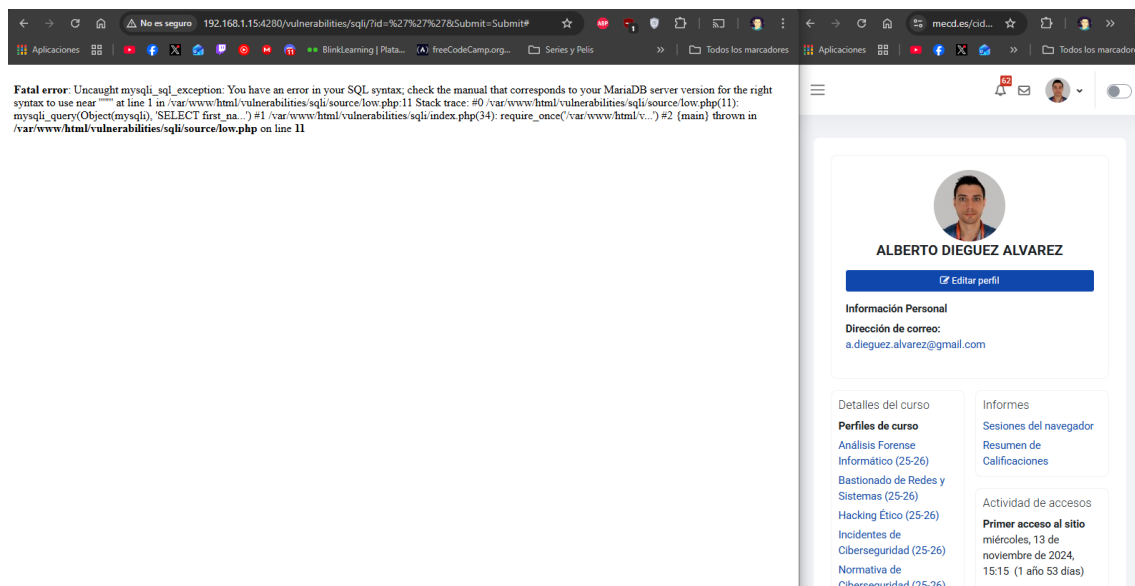
Pruebo con los IDs:





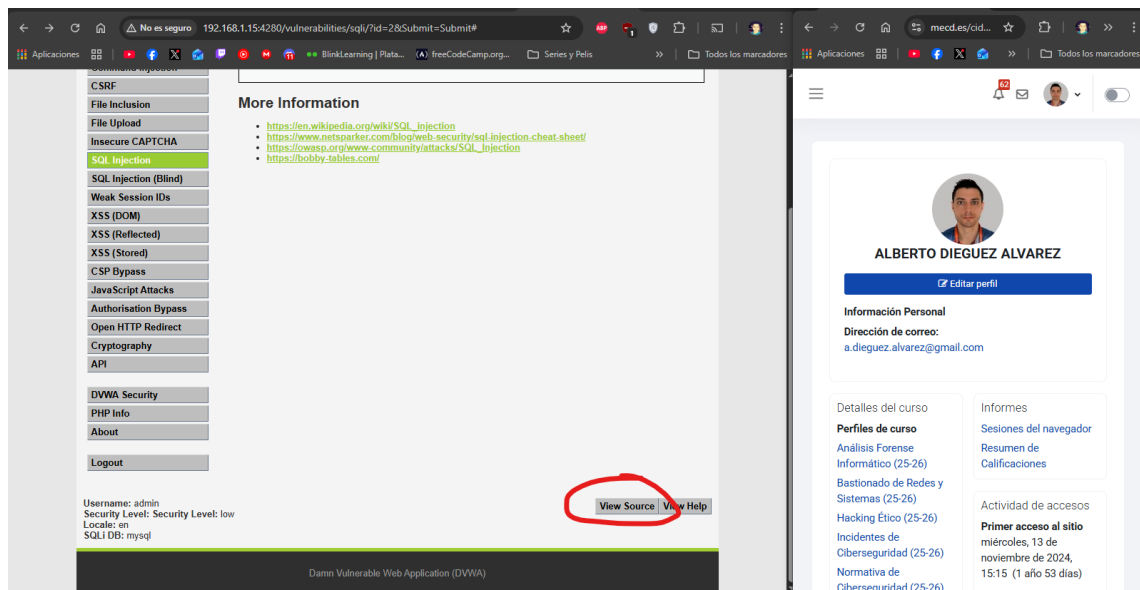
4. Trataremos de ver qué sucede cuando le damos resultados no esperados, por ejemplo, tres comillas simples '''

Nos da un error al ejecutar la consulta en la BBDD.

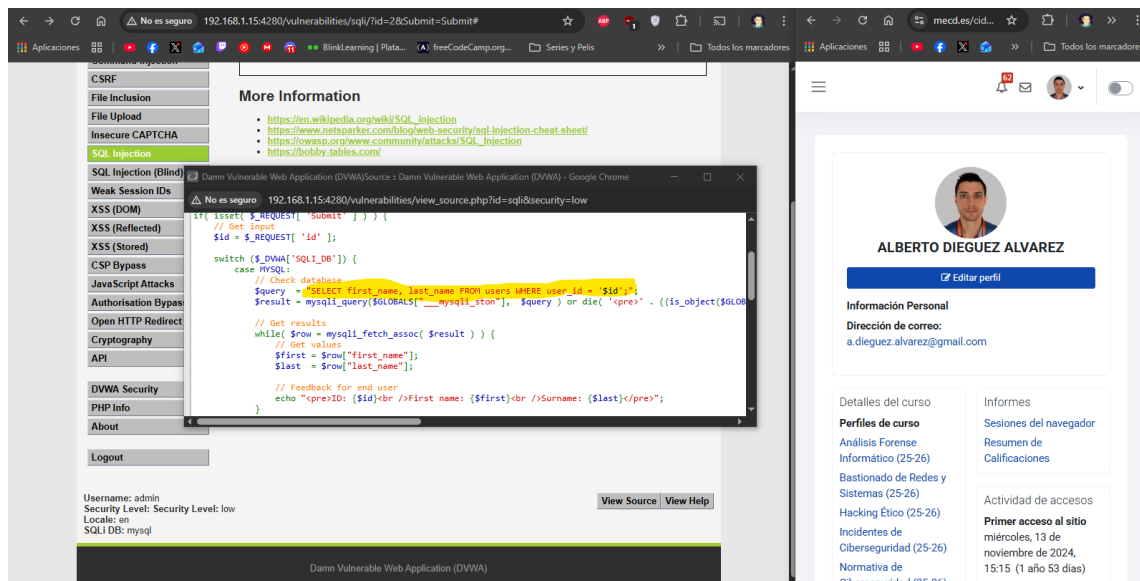


5. Revisaremos el código del aplicativo web para entender qué ha sucedido y que consulta (query) se ha intentado realizar. Para ver el código pulsar el **botón View Source** que hay al final de la página.

Podemos ver que al ejecutar la consulta se rompe la sintaxis y falla la query.

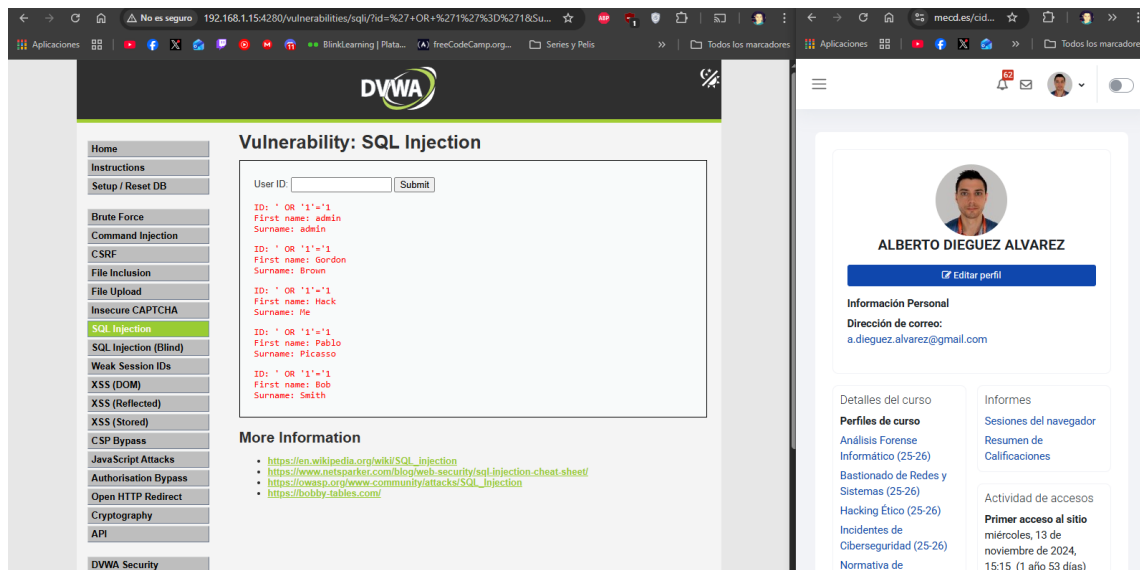


Y aquí vemos la consulta que hace:



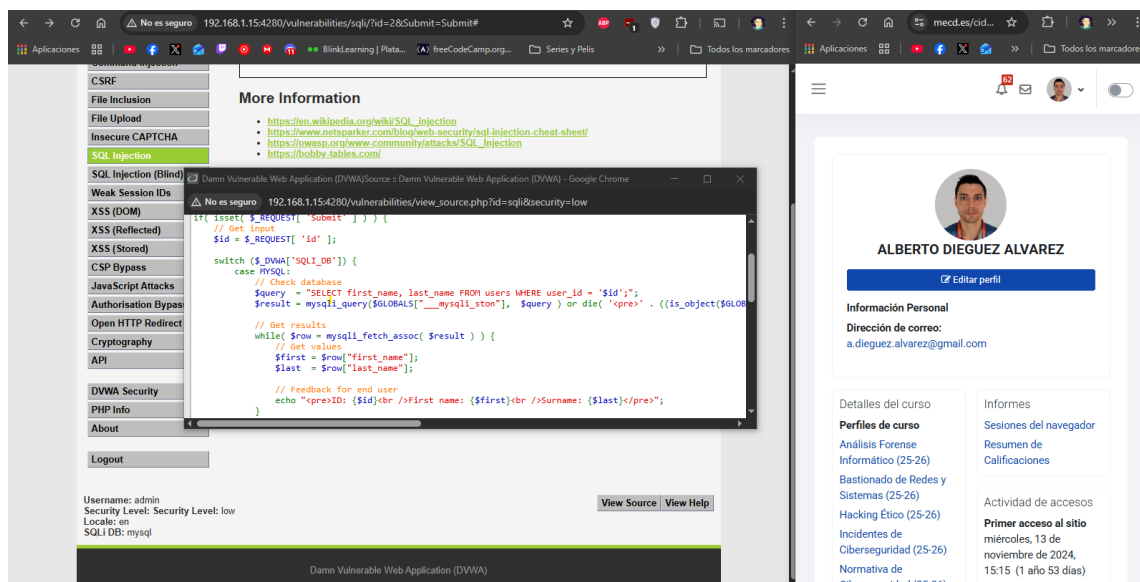
6. Probaremos con otros IDs como: ' OR '1'='1 (muy importante poner bien las comillas)

Ejecuto el comando y lo que hace es listar todos los IDs.



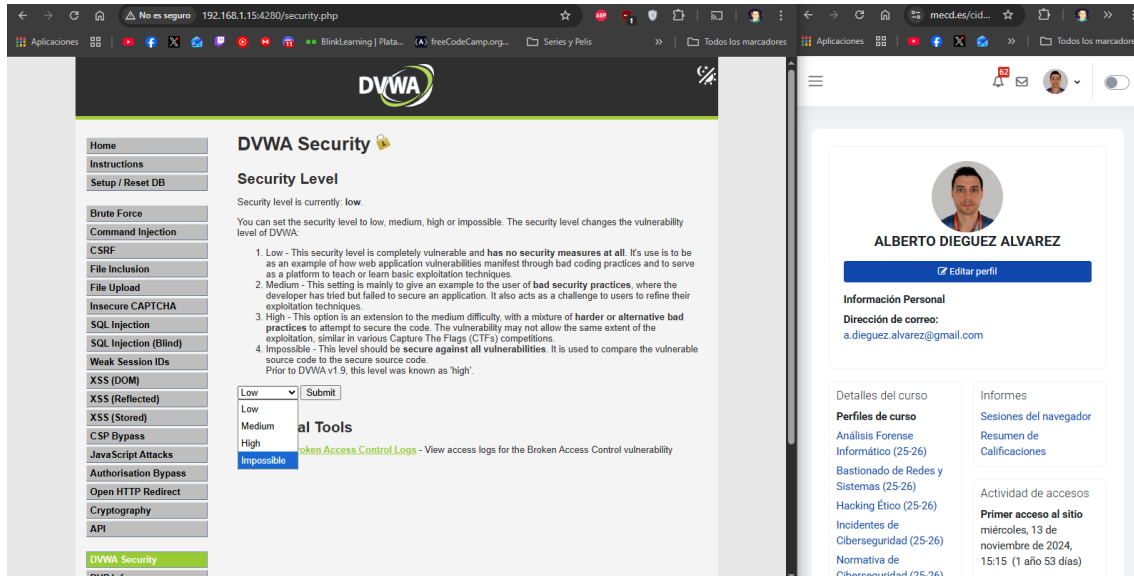
7. Revisaremos el código del aplicativo web para entender qué ha sucedido y que consulta (query) se ha realizado. Para ver el código pulsar el **botón View Source** que hay al final de la página.

Como ya hemos comprobado en la query, lo que estamos haciendo ahora es con las comillas simples hacer que se cumpla la primera condición o que 1=1 que se cumple siempre, entonces ejecuta la query.

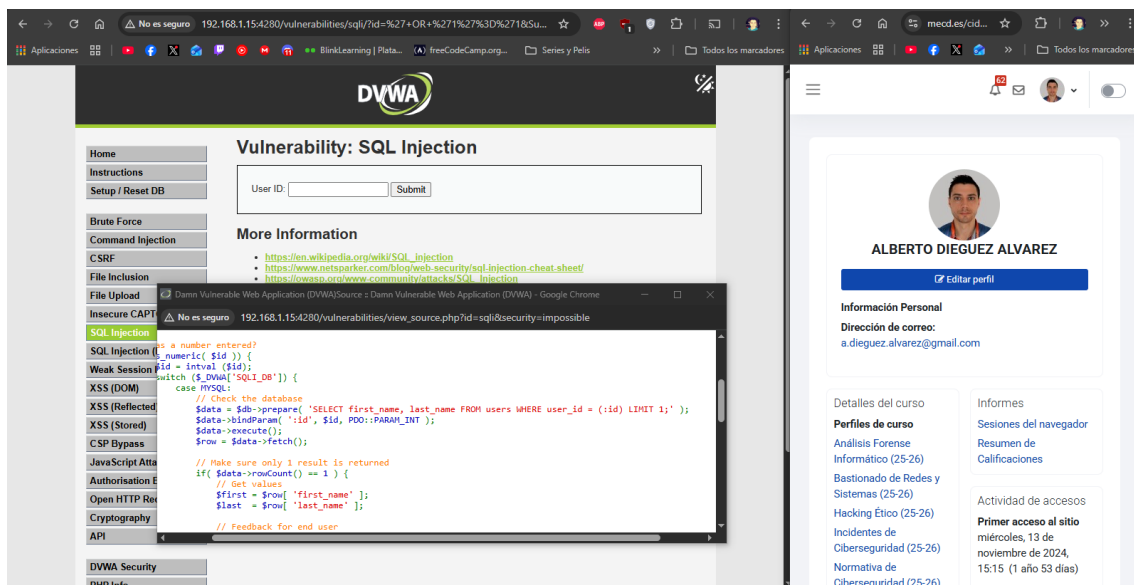


8. Probaremos a cambiar el modo de seguridad de DVWA a **Impossible** (dónde no debería haber vulnerabilidades) y volveremos a realizar todos los pasos de este apartado: probar con un id de usuario, probar con tres comillas, probar con ' OR '1'='1 y revisar el código del aplicativo.

Activo el modo imposible.



Ejecuto el comando y esta vez no devuelve nada, pero podemos ver como esta construida la query y que está bien optimizada y parametrizada. Es lo mismo para las tres comillas que para el OR, el resultado es el mismo.



Apartado 4: Preguntas finales

1. Indica cómo se ha comportado el aplicativo web:

- En el caso 3 comillas simples en el modo **Low**

Como hemos visto en las fotos anteriores en modo Low esta es la query:

```
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id';";
```

Al insertar las "" la query quedaría:

```
$query = "SELECT first_name, last_name FROM users WHERE user_id = ''''';";
```

Entonces esto rompe la query y da el error que hemos visto.

- En el caso ' OR '1'='1 en el modo **Low**

Para este caso la query quedaría así:

```
$query = "SELECT first_name, last_name FROM users WHERE user_id = '' OR '1'='1';";
```

Como vemos aquí nos dice que o user_id es empty o que se cumple 1=1, porque hemos insertado código que se ejecuta en la query a la BBDD.

- En el caso 3 comillas simples en el modo **Impossible**
- En el caso ' OR '1'='1 en el modo **Impossible**

Comparando las URLs de las peticiones, primera Low y segundo Impossible.

<http://192.168.1.15:4280/vulnerabilities/sqli/?id=%27+OR+%271%27%3D%271&Submit=Submit#>

http://192.168.1.15:4280/vulnerabilities/sqli/?id=%27+OR+%271%27%3D%271&Submit=Submit&user_token=5e5791f587f767c256d46b23e6445ab0#

Vemos que la segunda tiene user_token y como vimos en las imágenes anteriores las peticiones son diferentes, para imposible la consulta está parametrizada y es más robusta.

2. Indica qué consulta exacta (el código SQL) crees que se ha intentado/se ha lanzado en la base de datos:

- En el caso 3 comillas simples en el modo **Low**

Como he comentado antes:

```
SELECT first_name, last_name FROM users WHERE user_id = ''';
```

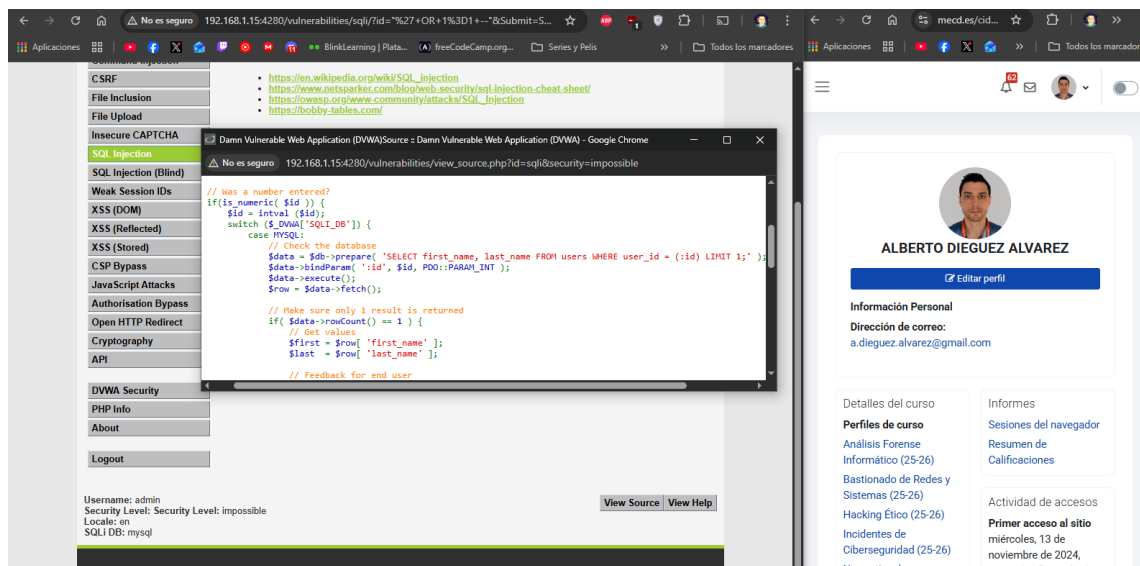
- En el caso ' OR '1'='1 en el modo **Low**

Como he comentado antes:

```
SELECT first_name, last_name FROM users WHERE user_id = '' OR '1'='1';
```

- En el caso 3 comillas simples en el modo **Impossible**

Como vemos en el código, se asegura que id es numérico y además está escapado y no se ejecuta hasta \$data->execute();



Como id está escapado, la query quedaría así.

```
SELECT first_name, last_name FROM users WHERE user_id = '' LIMIT 1;
```

- En el caso ' OR '1'='1 en el modo **Impossible**

En este caso quedaría así:

```
SELECT first_name, last_name FROM users WHERE user_id = '' OR '1'='1' LIMIT 1;
```

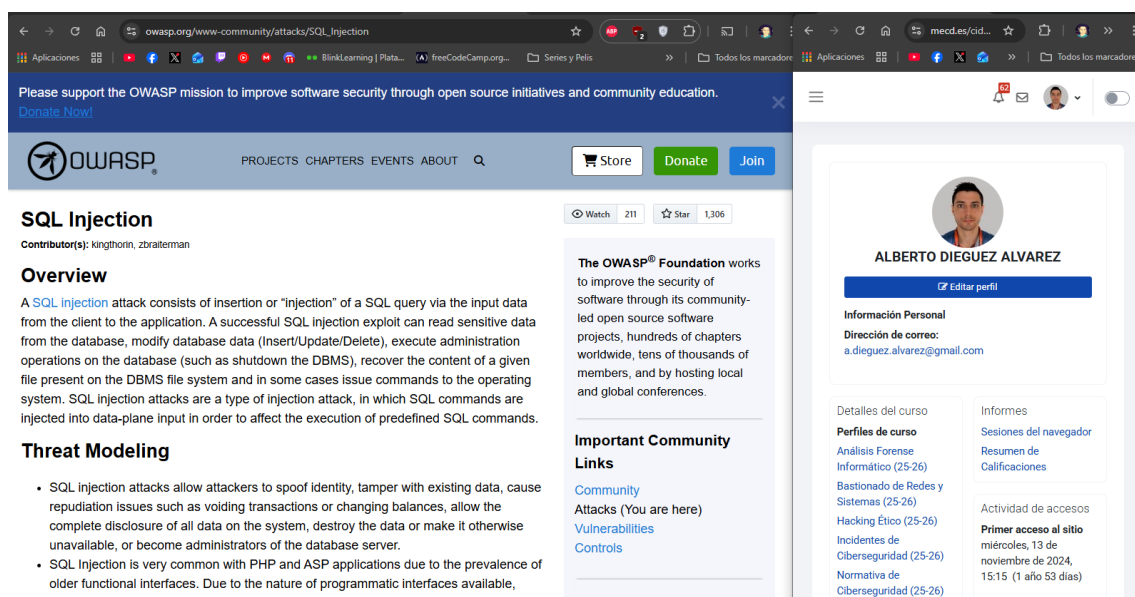
Como vemos en este modo escapa los valores.

3. Realiza una comparativa del código php del modo Low y del modo Impossible ¿Qué diferencias ves?

Como se ha explicado anteriormente y en las imágenes en el código podemos ver que en Low no se escapan ni se comprueba el tipo de valor, mientras que, en imposible, la consulta se prepara y se ejecuta, es decir se comprueba el tipo de valor, se escapan caracteres y se ejecuta la consulta.

4. Revisa la entrada de OWASP para este tipo de vulnerabilidad (https://owasp.org/www-community/attacks/SQL_Injection). ¿Cómo crees que afecta el código php del nivel low y del nivel impossible a la vulnerabilidad que estábamos explotando?

Reviso la entrada:



The screenshot shows the OWASP website's page for SQL Injection. The main content area includes an overview of SQL injection attacks, threat modeling, and important community links. A sidebar on the right displays the profile of Alberto Dieguez Alvarez, including his email address and a list of courses he has completed.

SQL Injection
Contributor(s): kingthorin, zbratterman

Overview
A [SQL Injection](#) attack consists of insertion or "injection" of a SQL query via the input data from the client to the application. A successful SQL injection exploit can read sensitive data from the database, modify database data (Insert/Update/Delete), execute administration operations on the database (such as shutdown the DBMS), recover the content of a given file present on the DBMS file system and in some cases issue commands to the operating system. SQL injection attacks are a type of injection attack, in which SQL commands are injected into data-plane input in order to affect the execution of predefined SQL commands.

Threat Modeling

- SQL injection attacks allow attackers to spoof identity, tamper with existing data, cause repudiation issues such as voiding transactions or changing balances, allow the complete disclosure of all data on the system, destroy the data or make it otherwise unavailable, or become administrators of the database server.
- SQL Injection is very common with PHP and ASP applications due to the prevalence of older functional interfaces. Due to the nature of programmatic interfaces available,

Important Community Links

- [Community](#)
- [Attacks \(You are here\)](#)
- [Vulnerabilities](#)
- [Controls](#)

Alberto Dieguez Alvarez
[Editar perfil]

Información Personal
Dirección de correo: a.dieguez.alvarez@gmail.com

Detalles del curso

Perfiles de curso

- [Análisis Forense Informático \(25-26\)](#)
- [Bastionado de Redes y Sistemas \(25-26\)](#)
- [Hacking Ético \(25-26\)](#)
- [Incidentes de Ciberseguridad \(25-26\)](#)
- [Normativa de Ciberseguridad \(25-26\)](#)

Informes

- [Sesiones del navegador](#)
- [Resumen de Calificaciones](#)

Actividad de accesos

Primer acceso al sitio
miércoles, 13 de noviembre de 2024, 15:15 (1 año 53 días)

Leyendo un poco la web, podemos concluir que una inyección SQL ocurre cuando datos que no son fiables se utilizan para modificar la consulta establecida alterando el correcto funcionamiento de la consulta.

Diferenciando entre Low – Impossible:

Low:

- No escapa caracteres
- No usa prepare

Impossible:

- Escapa caracteres
- Las variables de entrada son tratadas como datos, no pueden alterar la consulta
- Usan prepare y execute